Manual del Desarrollador — Sistema de Asistencia

Versión del sistema: 1.0.0

Versión del documento: 1.0

Fecha: 21 de octubre de 2026

Equipo / integrante: Francisco Herrera

Repositorio: `https://github.com/franciscoherrera-coder/proyecto.git`

Commit documentado: `6546130aabe5acd044538b0d9bd490409fb68d5a`

Control de versiones

Versión	Fecha	Responsable	Alcance
1.0	21/10/2026	Francisco Herrera	Línea base técnica del módulo de asistencia.

Índice

1. Definición general
2. Requerimientos y alcance
3. Arquitectura y módulos
4. Stack y decisiones
5. Modelo de datos
6. Procesos y servicios
7. Rutas y documentación técnica
8. Entorno, instalación y despliegue
9. Pruebas
10. Seguridad y datos sensibles
11. Mantenimiento y trabajo futuro
12. Conclusiones
13. Referencias y uso de IA
14. Anexos

1. Definición general

El módulo de asistencia integra en el sitio Laravel del ISFT N.º 38 la gestión de usuarios, carreras, materias, asignaciones, planillas diarias, códigos QR e indicadores de asistencia. Se construyó para reemplazar planillas dispersas, evitar duplicados y aplicar permisos por rol y ámbito académico.

Usuarios: directora de asistencia, preceptores, profesores y alumnos. Este documento está dirigido a desarrolladores PHP/Laravel, responsables de base de datos, despliegue y soporte.

2. Requerimientos y alcance

Requisitos funcionales verificables

- **RF-01 Autenticación:** ingresar por DNI o correo y contraseña; regenerar la sesión; cerrar sesión de forma segura.
- **RF-02 Roles:** resolver `is_admin`: 0 alumno, 1 administrador (directora o preceptor), 2 profesor.
- **RF-03 Ámbito:** limitar al preceptor a sus carreras y al profesor a sus materias.
- **RF-04 Usuarios:** registrar alumnos; la directora crea preceptores y asigna carreras; el administrador valida profesores.
- **RF-05 Alumnos:** crear, editar, eliminar y vincular registro académico con cuenta por DNI.
- **RF-06 Asignaciones:** vincular alumnos a materias y profesores a horarios/materias.
- **RF-07 Planilla:** guardar un estado único por materia, alumno y fecha; permitir edición solo para hoy y el día de cursada.
- **RF-08 QR:** generar token aleatorio, elegir presente/tarde, invalidar códigos previos y admitir solo alumnos asignados.
- **RF-09 Indicadores:** tardanza = 0,5 presencia; justificada = 0; mostrar historial y porcentajes.
- **RF-10 Alertas:** mínimo 80 % para prácticas y 70 % para el resto.
- **RF-11 Salidas:** filtrar porcentajes e imprimir planillas/listados.
- **RF-12 Configuración:** editar carrera y materia desde el panel administrativo.

Requisitos no funcionales

- Aplicación web responsive, interfaz en español y compatibilidad con navegadores actuales.
- Integridad mediante claves foráneas, índice único de asistencia y transacciones en operaciones compuestas.
- Contraseñas con hash de Laravel, CSRF en formularios y sesión regenerada después del login.
- Separación MVC y migraciones reproducibles.

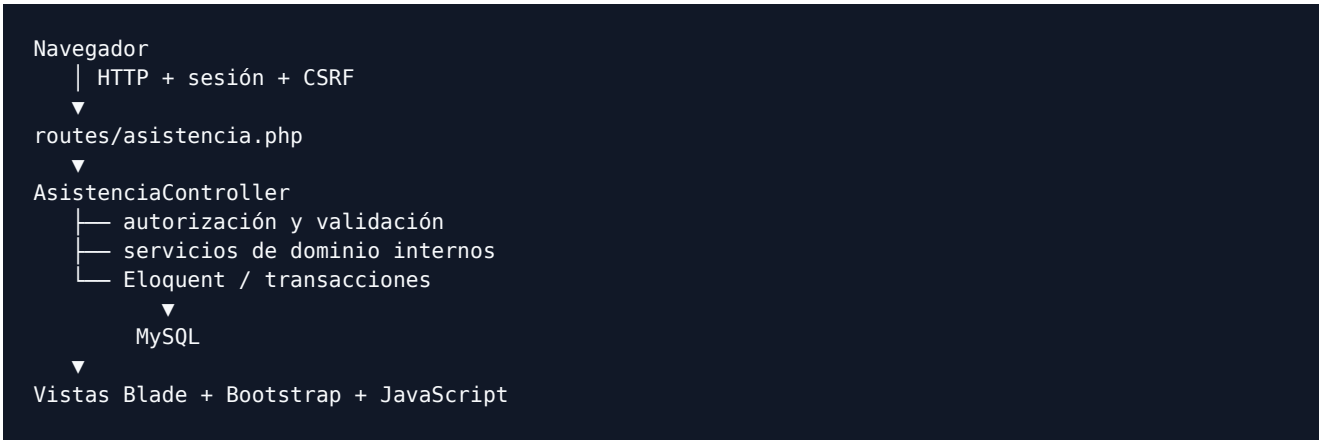
- Credenciales fuera del repositorio mediante `.env`.

Alcance y origen

Incluye el módulo bajo `/asistencia` y reutiliza entidades preexistentes del portal institucional (`users`, `registros`, `carreras`, `materias`, `profesors`, `horarios`, `anios`). Deriva de un proyecto Laravel previo y agrega tablas y relaciones de asistencia. Quedan fuera: API pública, app móvil nativa, carga de certificados, notificaciones automáticas, biometría y edición histórica desde UI.

3. Arquitectura y módulos

Es un monolito Laravel con patrón MVC y renderizado del lado servidor mediante Blade.



Componentes

- **Rutas:** `routes/asistencia.php`; declara las 31 operaciones web del módulo. Es incluido por `routes/web.php`.
- **Controlador:** `app/Http/Controllers/AsistenciaController.php`; concentra casos de uso, validación, autorización y consultas. No debe usarse como API JSON.
- **Modelos:** `AsistenciaDiaria`, `CodigoQrAsistencia`, `User`, `Registro`, `Materia`, `Horario`, `Profesor`, `Carrera`, `Anio`.
- **Vistas:** `resources/views/frontend/asistencia/`; acceso/panel, materia por rol, planilla, porcentajes, historial, perfil y listados.
- **Migraciones:** `database/migrations/2026_*`; crean asistencias, QR, pivotes de carreras y adaptaciones de usuario.
- **Front-end externo:** Flatpickr para fechas, QRCode.js para generar códigos y Html5Qrcode para escaneo desde cámara. Su disponibilidad depende de CDN.

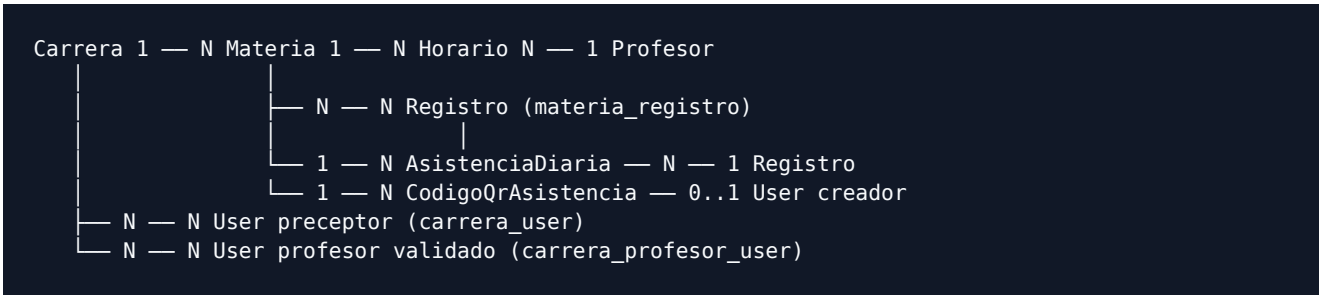
Dependencias internas relevantes: una materia debe existir en `horarios` para aparecer; el profesor se asocia a materia mediante horario; un alumno se asocia mediante `materia_registro`; directora se distingue por el correo técnico `admin.asistencia@isft38.test`.

4. Stack y decisiones de diseño

- PHP compatible declarado: `^7.3|^8.0`; entorno local encontrado: PHP 8.3.30 de Laragon.
- Laravel `^8.75`; Blade y Eloquent.
- MySQL/MariaDB con InnoDB.
- Bootstrap 4.6, jQuery 3.6, Laravel Mix 6, Sass; Node local 18.8.0 y npm 8.18.0.
- Laravel Sanctum está instalado, pero el módulo usa autenticación web de sesión.
- mPDF 8.2 y Dompdf están disponibles en el proyecto; este módulo imprime con CSS del navegador.

Se eligió un controlador único para integrar rápido con el portal heredado y reutilizar su sesión/modelos. La desventaja asumida es su tamaño y acoplamiento. Se prefirió QR con token persistido frente a URL firmada para poder habilitar/inhabilitar explícitamente. Se usa `updateOrCreate` e índice único para idempotencia. Alternativas no implementadas: servicios de dominio separados, políticas Laravel, SPA/API y proveedor QR del servidor.

5. Modelo de datos



Entidades propias y restricciones

- `asistencias_diarias`: `id` bigint PK; `materia_id` y `registro_id` FK con cascada; `fecha` date; `estado` varchar(20), valores admitidos por aplicación `presente|ausente|tarde|justificado`; `motivo_justificacion` nullable, `enfermedad|trabajo`; timestamps. Único `(materia_id, registro_id, fecha)` e índice `(materia_id, fecha)`.
- `codigos_qr_asistencia`: `materia_id` FK; `user_id` nullable FK; `fecha`; `token` varchar(64) único; `tipo` `presente|tarde`; `habilitado` boolean; timestamps e índice materia/fecha.
- `materia_registro`: pivote único alumno-materia con timestamps.
- `carrera_user`: pivote único preceptor-carrera. La sincronización garantiza un responsable activo por carrera.
- `carrera_profesor_user`: pivote único de profesor validado por carrera.

Entidades reutilizadas

- `users`: cuenta de acceso; nombre, correo único, contraseña hasheada, DNI incorporado por migración, carrera heredada e `is_admin`.
- `registros`: ficha personal/académica; el módulo usa principalmente nombre, apellido,

DNI, CUIL, correo y carrera. DNI/CUIL se validan como únicos en aplicación.

- `materias`: descripción, carrera, año, orden y relaciones académicas.
- `horarios`: materia, profesor, carrera, año y `dia` ISO 1=lunes … 7=domingo.
- `carreras`, `anios`, `profesors`: catálogos académicos.

Entradas: formularios, fecha y escaneo QR. Datos internos: roles, asignaciones, tokens y estados. Salidas: vistas, mensajes, indicadores y documentos impresos por navegador.

6. Procesos y servicios

Autenticación

```
validar identificador y contraseña
si identificador es numérico y users.dni existe → buscar por dni
si no → buscar por email
Auth::attempt → regenerar sesión → redirigir al panel
```

Guardado de planilla

Entrada: `fecha` y mapa `asistencias[registro_id][estado,motivo]`. Verifica profesor/materia, fecha no futura, día permitido, fecha igual a hoy, motivo obligatorio para justificado y pertenencia de cada alumno. En transacción ejecuta `updateOrCreate`. Salida: redirección con confirmación.

Registro por QR

Entrada: token de 64 caracteres. Busca código; exige habilitado y fecha de hoy; si no hay sesión redirige al login; exige rol alumno y asignación a materia. Actualiza o crea asistencia con el tipo del QR. La operación es idempotente por clave única.

Cálculo

Para cada fecha con planilla en la materia, la falta de fila individual se interpreta como ausencia automática. `presencias = presentes + tardanzas × 0,5`; `porcentaje = presencias / clases_totales × 100`. Justificadas se informan por motivo, sin sumar presencia.

Asignación administrativa

La directora sincroniza preceptores/carreras dentro de una transacción y reemplaza responsables anteriores. El preceptor solo opera en `carreraIdsUsuario`. Un profesor debe estar validado en la carrera antes de recibir materias.

7. Rutas y documentación técnica

Todas son rutas web, devuelven HTML/redirección y usan sesión/CSRF. No existe una API REST

pública del módulo.

Acceso y cuentas

- `GET /asistencia` → `index(Request)`: panel o formulario; `carrera_id` opcional. Error: acceso vacío/configuración faltante se expresa en la vista.
- `POST /asistencia/login` → `login`: `identificador`, `password`, `remember?=false`; 302 o validación 422.
- `POST /asistencia/registro` → `registrar`: alumno; nombre, apellido, email, password+confirmation, DNI, CUIL, carrera opcional.
- `POST /asistencia/logout` → `logout`: invalida sesión y token CSRF.
- `POST /asistencia/admin/usuarios` → `registrarDesdeAdmin`: solo directora; crea preceptor con `carrera_ids[]`.
- `PUT /asistencia/admin/preceptores/{user}/carreras` → `actualizarCarrerasPreceptor`: solo directora; arreglo no vacío y existente.

Administración

- `POST /asistencia/admin/alumnos` y `PUT /asistencia/admin/alumnos/{registro}`: alta/edición; nombre/apellido máx. 100, DNI/CUIL enteros únicos, email válido, carrera, contraseña mínima 8 y confirmada (opcional al editar).
- `DELETE /asistencia/admin/alumnos/{registro}`: elimina registro y acceso; 403 fuera de carrera.
- `POST /asistencia/admin/usuarios/profesor`: `user_ids[]`, `carrera_id`; valida candidato; 403 si no corresponde.
- `POST /asistencia/admin/profesores/materias`: `user_id`, `materia_ids[]`; exige validación del profesor y ámbito.
- `POST /asistencia/admin/profesor`: `materia_id`, `profesor_id?`; actualiza horario.
- `POST|DELETE /asistencia/admin/alumno`: `materia_id`, `registro_id`; agrega/quita pivote.
- `GET /asistencia/admin/materias/{materia}` y `/alumnos/{registro}`: detalle; 403 otra carrera, 404 alumno no asignado.
- `PUT /asistencia/admin/carreras/{carrera}`: descripción obligatoria; años, resolución, texto y carpeta opcionales.
- `PUT /asistencia/admin/materias/{materia}`: descripción, carrera/año/profesor opcionales y orden.

Profesor

- `GET /asistencia/profesor/materias/{materia}`: gestión de alumnos; 403 sin asignación.
- `POST /asistencia/profesor/alumnos`: `materia_id`, `registro_ids[]`; todos deben pertenecer a la carrera.
- `DELETE /asistencia/profesor/materias/{materia}/alumnos/{registro}`: quita asignación; 404 si no existe.
- `GET .../listado`: vista imprimible; `GET .../porcentajes` y `/porcentajes/{registro}`: resumen/detalle.

- `GET|POST .../planilla`: `fecha?` al leer; al guardar, `fecha` y `asistencias`; errores 403/422 según pertenencia, día y estado.
- `POST .../qr`: `fecha`, `tipo=presente|tarde`; solo hoy.
- `PATCH .../qr/{codigoQr}`: alterna habilitación; 404 si pertenece a otra materia o fecha.
- `GET /asistencia/profesor/alumnos/{registro}`: perfil limitado a alumnos relacionados.

Alumno/QR

- `GET /asistencia/alumno/materias/{materia}`: historial; 403 si no está asignado.
- `GET /asistencia/qr/{token}`: registra asistencia; 404 token inexistente; 403 inhabilitado, rol incorrecto o sin asignación.

CLI de Laravel: `php artisan`. Rutas: `php artisan route:list --path=asistencia`. No hay documentación OpenAPI ni tarea automática; este manual se actualiza desde rutas, controlador, modelos y migraciones.

8. Entorno, instalación y despliegue

Instalación limpia

```
git clone https://github.com/franciscoherrera-coder/proyecto.git
cd proyecto
composer install
copy .env.example .env
php artisan key:generate
# configurar DB_* y APP_URL en .env
php artisan migrate
npm install
npm run production
php artisan storage:link
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

En Windows/Laragon, seleccione PHP 8.x, configure el DocumentRoot a `public/` y cree una base MySQL/MariaDB con `utf8mb4`. Para desarrollo: `php artisan serve` y `npm run dev`.

Variables principales: `APP_NAME`, `APP_ENV`, `APP_KEY`, `APP_DEBUG`, `APP_URL`, `DB_CONNECTION`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`, sesión, caché, cola y correo. `.env` contiene secretos y no se versiona; producción debe usar `APP_DEBUG=false` y HTTPS.

Verificación: abrir `/asistencia`, crear/usar usuarios de prueba, comprobar `route:list`, ejecutar migraciones y revisar `storage/logs/laravel.log`. El repositorio conserva trabajo mediante Git en `main`; usar ramas y revisión antes de integrar.

9. Pruebas

Actualmente `tests/Feature/ExampleTest.php` y `tests/Unit/ExampleTest.php` son plantillas; no hay cobertura automatizada específica del módulo. Ejecutar:

```
php artisan test
# o vendor/bin/phpunit
```

Matriz mínima manual:

Caso	Entrada	Resultado esperado
Login válido/inválido	DNI/correo + clave	Panel / mensaje exacto de credencial
Ámbito preceptor	carrera propia/ajena	permitido / 403
Planilla hoy	cuatro estados	una fila única por alumno y fecha
Fecha histórica	POST anterior	validación 422
Justificado sin motivo	estado justificado	validación 422
QR válido/repetido	alumno asignado	crea/actualiza sin duplicar
QR inhabilitado/antiguo	token existente	403
Porcentaje	2 presentes, 1 tarde, 1 ausente	62,5 %
Sustitución de preceptor	carrera ocupada	reemplazo transaccional confirmado

Pendiente automatizar autorización, concurrencia QR, integridad de borrados, umbrales, nombres duplicados de profesor y compatibilidad entre esquemas heredados.

10. Seguridad y datos sensibles

- Laravel `web` aporta cookies cifradas, sesión, CSRF y route-model binding.
- Contraseñas se almacenan con `Hash::make`; nunca registrar ni copiar claves reales.
- El token QR usa `Str::random(64)`, es único, revocable y de fecha diaria.
- Autorización se aplica dentro del controlador; las rutas de asistencia no están agrupadas globalmente bajo `auth`, porque login, registro y QR necesitan acceso inicial. Cada operación sensible debe mantener su guardia explícita.
- Datos personales: nombre, apellido, DNI, CUIL, correo, carrera y ficha extensa de `registros`; profesores/preceptores ven solo el ámbito necesario. Aplicar minimización, copias cifradas, HTTPS, retención institucional y control de exportaciones.
- Riesgo conocido: la directora se identifica por un correo fijo. Conviene migrar a rol/permiso persistido.
- No incluir `.env`, volcados con datos reales, logs o contraseñas en Git. El archivo `dev338.sql` debe tratarse como sensible y anonimizarse antes de compartir.

11. Mantenimiento y trabajo futuro

Prioridades:

1. Dividir `AsistenciaController` en Requests, Policies y servicios de dominio.
2. Agregar pruebas Feature/Unit y CI.
3. Reemplazar el correo especial de directora por permisos configurables.
4. Unificar el modelo de roles heredado; otros módulos usan valores 3/4 y guardas distintas.
5. Crear flujo auditado de rectificación histórica.
6. Servir dependencias de QR/fecha localmente o aplicar SRI/CSP.
7. Agregar auditoría de altas, bajas, cambios y QR.
8. Normalizar email/DNI/CUIL con índices únicos de base de datos.
9. Reducir la asociación de profesores por coincidencia de nombre; usar una FK explícita.
10. Revisar `routes/web.php`: actualmente incluye grupos del portal sin middleware administrativo global.

Partes frágiles: compatibilidad condicional mediante `Schema::hasTable/hasColumn`, vínculo usuario-registro por DNI, vínculo usuario-profesor por nombre normalizado, dependencia de horarios y sustitución de preceptores. No modificar sin datos de prueba y respaldo.

12. Conclusiones

La dificultad central fue integrar asistencia a un esquema académico heredado con distintas convenciones de rol y relaciones indirectas. Se adoptaron comprobaciones de esquema, transacciones, pivotes y autorización por ámbito para mantener compatibilidad. Se recortaron API, app móvil, certificados y edición histórica para concentrar la entrega en el circuito diario. El principal aprendizaje es que roles y vínculos de identidad deben modelarse explícitamente para reducir complejidad futura.

13. Referencias y uso de IA

- Documentación de Laravel 8: routing, authentication, validation, Eloquent, migrations y CSRF.
- Documentación de Bootstrap 4.6, Flatpickr, QRCode.js y Html5Qrcode.
- Código fuente, migraciones, vistas, `composer.json`, `package.json` y repositorio del proyecto.
- Guía rápida de contenidos suministrada para estructurar ambos manuales.

Se utilizó un asistente de IA para inspeccionar el código, organizar la documentación, redactar diagramas textuales y generar los archivos. Las afirmaciones técnicas se contrastaron con el repositorio; deben validarse nuevamente en el ambiente productivo y con las políticas institucionales.

14. Anexos

Archivos clave

```
routes/asistencia.php
app/Http/Controllers/AsistenciaController.php
app/Models/AsistenciaDiaria.php
app/Models/CodigoQrAsistencia.php
resources/views/frontend/asistencia/
database/migrations/2026_08_26_000001_create_asistencias_diarias_table.php
database/migrations/2026_09_03_000001_create_codigos_qr_asistencia_table.php
database/migrations/2026_09_17_000001_create_carrera_user_table.php
database/migrations/2026_09_24_000001_create_carrera_profesor_user_table.php
```

Respaldo y recuperación

Antes de migrar o corregir datos: detener escrituras, generar dump cifrado, verificar restauración en entorno separado y registrar commit/migración. No editar porcentajes: se calculan desde `asistencias_diarias`. Toda corrección debe modificar el registro fuente conservando auditoría externa hasta incorporar historial de cambios.

Checklist de entrega

- `.env` productivo fuera de Git; `APP_DEBUG=false`; HTTPS.
- Migraciones aplicadas y respaldos probados.
- Directora, preceptores y carreras revisados.
- Horarios con día/profesor y alumnos asignados.
- Prueba de planilla, QR, porcentaje e impresión.
- Canal de soporte y URL productiva incorporados al Manual de Usuario.